QA_Automation_KC 동작 흐름

URL 또는 기획서를 넣으면 테스트 케이스를 자동으로 만들고 실행해서 결과를 보여주는 프로그램 · v0.013 기준

1. 한눈에 보기

STEP 1

입력

URL · 기획서(PDF 등) · 테스트 범위 · 브라우저 · 로그인 · 동작 확인 범위

로그인이 필요한 사이트
로그인 창에서 사람이 직접 로그인 (ID/PW · SNS · 인증) → 로그인 상태를 저장해 이후 모든 단계에 사용

▼

STEP 2 · 크롤러

페이지 찾기

시작 페이지 → 세부 페이지 → 그 세부 페이지 … (같은 사이트만, 최대 50페이지)

다른 사이트 링크(광고 · 외부 링크)
페이지로 분석하지 않고, 누르면 그 사이트로 **이동하는지만** 확인

▼

STEP 3 · 하네스 generate-specs.js

페이지마다 화면 분석

버튼 · 링크 · 배너 · 입력창 · 검색창 · 드롭다운 · 표 · 문구 · 이미지 · 롤링 배너를 찾고, **실제로 눌러서** 무슨 일이 일어나는지 관찰

기획서를 넣은 경우
AI가 기획서로 TC를 쓰고, 그 TC를 STEP 3에서 찾은 화면 요소와 연결

▼

STEP 4

테스트 케이스 생성

화면 위→아래 순서로 요소마다 **노출 · 문구 · 동작** 확인 항목을 만들고, 규칙 기반 오타 의심 검사 · `cases.html` (실행 전) 저장

AI 테스트 버튼
페이지 화면 캡처 + 요소 목록을 AI에게 보내 기대 동작 TC와 화면 이슈(오타 · 레이아웃 등)를 추가

▼

STEP 5 · Playwright

실행 = 실제 사이트와 비교

Chrome · Safari(WebKit) · Firefox에서 새 브라우저로 사이트를 다시 열고, 분석 때 관찰한 결과대로 동작하는지 하나씩 확인 (진행률 표시 · 중지 가능)

비교 기준
① 분석 때 실제로 관찰한 동작
② 기획서에 적힌 기대 결과
③ AI가 판단한 기대 동작

▼

STEP 6

결과

앱 화면 결과 · `cases.html` (대분류 · 중분류 · 소분류 · 과정 · 기대결과 · 결과 · 이유) · Playwright 리포트(실패 스크린샷 · 영상) · `run.log`

결과 폴더에 실행마다 따로 저장
CSV(엑셀)로도 내려받기 가능

> 핵심 원리: 이 프로그램은 **사람이 하듯 화면을 직접 눌러 보고**(분석), 그때 본 결과를 **테스트 코드로 고정**한 뒤(생성), **새 브라우저에서 다시 해 봐서 같은 결과가 나오는지** 확인합니다(실행). 기획서나 AI를 쓰면 "관찰한 결과" 대신 "기대해야 하는 결과"와 비교할 수 있습니다.

2. STEP 1 · 입력

항목	내용
URL	테스트할 사이트 주소. 기획서만 넣으면 TC 작성까지만 진행
기획서 첨부	PDF · 이미지 · 문서. AI API Key(Gemini · Claude · ChatGPT 중 선택)가 필요
테스트 범위	**세부 사이트 모두**: 연결된 세부 페이지까지 / **입력한 페이지만**: 그 페이지 하나
브라우저	Chrome · Safari(WebKit 엔진) · Firefox. 여러 개 고르면 브라우저마다 실행하고 결과 차이를 따로 표시
로그인	URL을 넣으면 로그인 화면인지 자동 확인 → **로그인 창 열기**에서 직접 로그인 → **로그인 완료**. 사이트별로 이 PC에만 저장
동작 확인 범위	**안전(기본)**: 삭제 · 결제 · 제출 · 저장 · 출근처럼 데이터가 바뀌는 버튼은 누르지 않고 노출 · 문구만 확인 **전체 동작 확인**: 그런 버튼도 실제로 누름 (테스트 계정 · 테스트 서버 전용). 탈퇴 · 결제 · 송금 · 로그아웃은 항상 누르지 않음
실행 버튼	**규칙 기반 테스트**(AI 없이) / **AI 테스트**(규칙 기반 + AI 화면 분석)

3. STEP 2 · 페이지 찾기 (크롤러)

시작 페이지 → 링크 수집 → 1단계 세부 페이지 → 2단계 … → 새 페이지 없음 / 50개 도달

규칙	설명
같은 사이트	서브도메인 포함 (www.naver.com · news.naver.com 은 같은 사이트). 단, lovable.app · vercel.app 같은 호스팅 도메인은 서브도메인마다 다른 사이트
다른 사이트	광고 배너 · 외부 링크는 분석하지 않고 "누르면 그 사이트로 이동하는지"만 테스트
중복 제거	`/notice/1`, `/notice/2`처럼 ID만 다른 상세 페이지는 하나만 · 광고 추적값(`utm_…`)만 다른 주소는 같은 페이지
파일 링크	PDF · 이미지 · 압축 파일은 페이지로 분석하지 않음
접속 차단	봇 차단 · 보안 확인 화면이면 "접속 차단"으로 표시하고 접속 상태만 검사

4. STEP 3 · 페이지 분석 (하네스 generate-specs.js)

페이지마다 **새 브라우저**로 열고, 화면 구조(DOM · 위치 · 스타일)를 읽어 테스트할 대상을 찾은 뒤, 요소를 **실제로 눌러서** 결과를 관찰합니다. 특정 사이트에 맞춘 규칙 없이 화면 구조만으로 판단합니다.

3-1. 무엇을 찾는가

대상	찾는 방법	만들어지는 확인 항목
버튼 · 링크 · 탭	클릭 가능한 모든 요소	노출 · 문구 · 동작
배너 · 로고	넓은 영역에 여러 줄 문구/이미지를 담은 클릭 요소 · 헤더 왼쪽 위 이미지	노출 · 문구(줄마다) · 이미지 로드 · 동작
일반 문구 · 이미지	클릭 요소 밖의 텍스트 덩어리 · 대체텍스트가 있는 이미지	노출 (숫자는 형식만)
롤링 배너	몇 초간 9번 관찰해 보였다 사라지는 문구를 같은 영역끼리 묶음 (좌우 · 상하 · 페이드)	문구가 차례로 화면에 나타남
툴팁	마우스를 올렸을 때 요소 근처에 새로 나타나는 문구	마우스 오버 노출
입력창 · 검색창	입력 요소 · 이름에 "검색/조회/필터"가 있으면 검색창	입력 · 목록의 실제 값으로 검색 → 그 값만 표시 · 없는 값 → 결과 없음
드롭다운	선택 박스(select) · 누르면 옵션(option/menuitem)이 나타나는 버튼	옵션을 하나씩 선택(최대 8개) → 선택 값 표시 / 목록 변경 / 페이지 이동
표	머리글이 있는 표	열 구성 · 모든 행의 칸 수 · 열별 형식(날짜 · 시간 · 숫자) · 정렬 버튼
반복 목록	같은 목록 안 같은 위치 · 같은 종류 요소 3개 이상 (표의 행 버튼 · 달력 날짜 · 상세 링크)	대표 1개만 테스트 ("목록 N개 중 대표")

3-2. 눌러서 관찰하는 결과 (동작 유형)

관찰 결과	판단 기준	테스트에서 확인하는 것
페이지 이동	주소가 바뀜	같은 주소로 이동 (외부 사이트는 도메인만)
새 탭 열기	팝업 창이 열림	새 탭이 열리고 해당 사이트로 감
모달 · 시트	화면 위에 고정된 새 층이 나타남	열림 → 닫기 버튼/ESC로 닫힘 (문구 개수가 늘었다 원래대로)
펼침 · 접힘	`aria-expanded` 변화	펼쳐졌다가 다시 누르면 접힘
탭 전환	`aria-selected` 변화	해당 탭이 선택됨
표 정렬	표 머리글 버튼 클릭 후 행 순서 변화	행 순서가 바뀜
콘텐츠 노출	화면에서 찾을 수 있는 새 문구가 나타남 (숫자만 바뀐 줄은 제외)	그 문구가 나타남
깨진 링크	브라우저 오류 페이지로 감	정상 페이지로 열리는지 (현재 상태면 실패로 보고)

클릭 안정성	눈에 보이는 변화 없음	눌러도 JS 오류가 없음

3-3. 오판을 막는 장치

상태 격리

페이지마다 새 브라우저로 분석하고, 클릭할 때마다 쿠키 · 저장소를 처음 상태로 되돌림. 예) 다른 페이지에서 접은 사이드바가 다음 분석에 섞이지 않음

바뀌는 값 처리

숫자가 든 문구(건수 · 날짜 · 시간)는 형식만 비교 · 알림 배지 숫자는 무시 · 시계/카운트다운 · 롤링 문구는 클릭 결과로 보지 않음

안전 모드

삭제 · 결제 · 제출 · 저장 · 출근 · 로그아웃 등은 누르지 않고 노출 · 문구만 확인. 팝업은 "확인" 대신 닫기 · 취소 · ESC로 닫음

찾기 쉬운 로케이터

Playwright 권장 순서: 역할+이름 → 라벨 → placeholder → 링크 주소 → 고유 속성 → 태그+문구. 이름 끝 숫자 배지는 "있어도 없어도" 찾음

5. STEP 4 · 테스트 케이스 생성

분석 결과 (analysis.json) → 화면 위→아래 정렬 → 요소마다 확인 항목 → Playwright 코드 (*.spec.ts) + cases.html (실행 전)

한 요소 = 한 케이스, 케이스 안에서 확인 항목을 **각각 따로** 판정합니다. 예) *베너 "USDT 결제, QR 하나면 끝!"*

확인 항목	내용
노출	화면에 보이는가 (실패하면 나머지 항목은 "확인 불가"로 건너뜀)
문구	요소 안 문구를 줄마다 확인 (숫자는 형식만)
이미지 로드	요소 안 이미지가 깨지지 않았는가
마우스 오버	툴팁 문구가 나타나는가
동작	3-2의 관찰 결과대로 동작하는가
오타 의심	실패가 아닌 **경고**. 사이트 전체 문구 비교(자주 쓰인 단어와 헷갈리는 자모 하나 차이) · 형식 오류(단어 반복 · 괄호 짝 · 낱자모) · 자주 틀리는 표기(되요 · 몇일 · 할께 …)

분석이 끝나면 결과 폴더에 `cases.json`과 `cases.html`(모든 항목 "미실행")이 먼저 저장됩니다.

6. STEP 5 · 실행 (사이트와 비교)

새 브라우저 → (저장된 로그인 적용) → 페이지 열기 · 팝업 처리 → 확인 항목 순서대로 실행 → 항목별 통과/실패 기록

설정	값	이유
동시 실행	4개	사이트에 요청이 몰리면 무더기 타임아웃 · PC 메모리 보호
재시도	1회	일시적 네트워크 지연. 재시도로 통과하면 "불안정"으로 표시
화면 · 언어	분석 때와 동일	1440×900, 분석한 PC의 언어 · 시간대로 고정해 문구가 달라지지 않게
진행률	앱 화면 프로그레스 바	분석: 페이지 수 · 실행: 끝난 케이스 / 전체 (통과 초록 · 실패 빨강)
중지	중지 버튼	실행 중인 분석 · 테스트와 브라우저를 즉시 종료

> **"사이트와 비교"의 의미** — 규칙 기반 테스트의 기대 결과는 **분석 때 실제로 관찰한 동작**입니다. 그래서 "지금 사이트가 분석 시점과 같은 화면 · 같은 동작인가"를 확인하는 회귀 테스트에 강합니다. "원래 이렇게 동작해야 한다"를 확인하려면 기획서(기획서의 기대 결과와 비교)나 AI 테스트를 함께 씁니다.

7. STEP 6 · 결과

실행마다 결과 폴더(기본: 문서/QA_Automation_KC, 앱에서 변경 가능) 아래 `날짜-시간` 폴더가 만들어집니다.

파일	내용
cases.html	TC 표: 대분류(페이지) · 중분류(요소) · 소분류(확인 항목) · 과정 · 기대결과 · 결과 · 이유. 검색 · 필터 · CSV(엑셀) 다운로드 · 화면 캡처 링크
report/	Playwright HTML 리포트. 실패한 케이스의 스크린샷 · 영상 · 트레이스
summary.json · results.json	결과 요약 / 원본 실행 결과
analysis.json · cases.json	분석 결과(페이지 · 요소 · 문구 · 오타 의심) / 케이스 목록
run.log	실행 전체 로그 (분석 실패 원인 확인용)
doc-testcases.* · ai-testcases.*	기획서 · AI 테스트를 사용한 경우의 TC와 결과

결과 읽는 법

결과	의미
통과	분석 때와 같게 동작
실패	다르게 동작. 이유 열에 어느 항목이 왜 실패했는지 표시. 실제 버그 · 사이트 변경 · 데이터 변화 중 무엇인지 사람이 판단
경고	오타 의심 (확정 아님)
건너뜀	안전 모드로 누르지 않음 · 노출 실패로 확인 불가 등 (이유 열 참고)
불안정	첫 시도는 실패, 재시도는 통과

8. 선택 기능

기획서 **URL + 기획서**

- AI가 기획서를 읽고 TC 작성
- TC마다 STEP 3에서 찾은 화면 요소와 연결
- 연결된 TC는 기획서의 기대 결과로 실행 · 못 찾은 TC는 "화면에서 못 찾음" · 판단 불가는 "수동 확인 필요"
- 기획서와 실제 동작이 다르면 "실제 동작 차이"로 표시

AI 테스트 **URL만**

- 페이지 화면 캡처 + 요소 목록을 AI에 전송
- AI가 기대 동작 기준 TC를 작성해 함께 실행
- 이미지 속 글자 · 맞춤법 · 어색한 문구 · 레이아웃 이슈 보고

로그인 **로그인이 필요한 사이트**

- 로그인 화면 자동 감지
- 사람이 직접 로그인 (어떤 방식이든)
- 저장된 로그인으로 분석 · 실행, 만료 시 다시 로그인 안내

전체 동작 확인 **테스트 계정 전용**

- 저장 · 등록 · 제출 · 삭제 버튼도 실제로 눌러 확인
- 분석 · 브라우저별 실행 · 재시도마다 반복해서 눌릴 수 있음

9. 알아둘 한계

- 이미지 안에 그려진 글자 · canvas 화면은 규칙 기반으로 읽지 못함 (AI 테스트로 보완)
- 숫자가 든 문구는 형식만 비교하므로 "10% → 7%" 같은 값 변경은 규칙 기반으로 잡지 못함 (기획서로 보완)
- 규칙 기반의 기대 결과는 "분석 시점의 동작"이라, 분석 시점에 이미 잘못된 동작은 정상으로 기록됨
- 마우스 오버로만 나타나는 하위 메뉴 · iframe 안 · Shadow DOM 안 요소는 아직 분석 대상이 아님
- 서버에 저장되는 설정(계정 설정 등)은 분석 후 되돌릴 수 없음 (전체 동작 확인을 켰을 때만 해당)